Contender 11: Self-Indexed Forest Reflection over Kind-Polymorphic System F_ω

A constructive large number and a kernel-checked comparison for BusyLean

Prepared for the BusyLean project

12 October 2026

Abstract

We propose *Contender 11* for BusyLean, a closed computable natural number that strictly exceeds the repository's current Contender 10. Contender 10 reflects once over a strongly normalizing, kind-polymorphic extension of System F_ω with a unary oracle. The new construction lifts that reflection to an indexed family of oracles, then iterates it over BusyLean's self-indexed well-founded tree: the tree joining every $W\mathbb{N}$ tree denoted by the repository's W-language. The final number is

$$C_{11} = F_{11}(F_{11}(42)), \qquad F_{11}(n) = \mathrm{stage}_{11}(\lambda_.\, F_{10}(F_{10}(\cdot)), \mathrm{index}, n).$$

The formal margin is substantially stronger than a terminal inequality. For every input n, $F_{11}(n)$ exceeds the whole generating family $F_{10}(F_{10}(n))$; every enumerated kind-polymorphic program at budget n under the root forest oracle is below $F_{11}(n)$; and every W-language-definable index tree is eventually dominated. The required theorem `contender_10_lt_contender_11` is accepted by Lean 4.34.0. A reconstructed verification unit containing the exact 30-module dependency closure of BusyLean commit `4c4b03d0e2003c479e3bee90a21c8bea13f66c40` plus the new definitions and proofs returned `okay = true`, an empty failed-declaration list, and the permitted axiom set {`propext`, `Quot.sound`}. The archive contains ready-to-copy Lean modules, audits, a clean-build timing script, a reproducible remote checker, and an integration patch.

Main certified result.

```
def family10 (n : Nat) : Nat := F10 (F10 n)
def F11 (n : Nat) : Nat := stage11 (fun _ => family10) index n
def contender_11 : Nat := F11 (F11 42)

theorem contender_10_lt_contender_11 :
    contender_10 < contender_11
```

The definition does not mention any earlier contender constant. Its dependency closure was mechanically checked to avoid `contender_5`, . . . , `contender_10` and the legacy proposal constants.

Contents

1 Repository baseline and design objective **2**

2 The three reusable ingredients **3**

2.1 Kind-polymorphic reflection 3

2.2 Packing a family into one oracle 3

2.3 The self-indexed W-tree 4

3 Definition of Contender 11 **4**
3.1 Family reflection at one node . . . 4
3.2 Reflection along a forest . . . 4
3.3 Seed and diagonal . . . 4

4 Formal domination theorems **5**
4.1 One-step inequalities . . . 5
4.2 Child-to-parent growth . . . 5
4.3 Pointwise domination of the predecessor family . . . 6
4.4 The required comparison . . . 6
4.5 Program and tree domination . . . 6

5 Why the number is computable in principle **7**

6 Size analysis: formal facts and heuristic calibration **7**
6.1 What is formally established . . . 7
6.2 A deliberately cautious heuristic . . . 7

7 Verification and audit record **8**
7.1 Remote kernel/elaborator check . . . 8
7.2 What this check does and does not establish . . . 8

8 Engineering choices and rejected alternatives **9**
8.1 Why explicit inclusion beats monotonicity . . . 9
8.2 Why reuse the W-language index . . . 9
8.3 Why not jump immediately to a kind-universe ladder . . . 9
8.4 Why not use the incumbent constant . . . 9

9 Further research questions **9**

10 Reproducibility and archive layout **10**

11 Conclusion **11**

A Complete Lean definition module **11**

B Complete Lean comparison module **12**

1 Repository baseline and design objective

This proposal targets BusyLean commit `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`. At that snapshot the current entry is Contender 10:

```
def family9 (n : Nat) : Nat := F9 (F9 n)
def F10 (n : Nat) : Nat := Fk.reflectFk family9 n
def contender_10 : Nat := F10 (F10 42)
```

The object language, called `Fk` in the source, is System F_ω with kind variables, kind abstraction and application, and type quantification over closed kinds. Its checker is sound, and its erased terms are normalized by a total function justified by the formal theorem `Fk.sn_of_typed`. Consequently

`Fk.fvalue f n t` is a total natural-number function for every oracle f, budget n, and Church-style program t.

The contest imposes four constraints that shape the construction.

C1. The number must be a closed, computable `Nat`; no Busy Beaver, noncomputable choice, unsafe implementation, partial definition, or opaque stand-in is permitted.

C2. The comparison with the incumbent must be a formal theorem.

C3. Only `propext` and `Quot.sound` may occur in the axiom closure.

C4. The new definition must use the predecessor's *construction*, not merely quote the constant `contender_10` and add one.

A useful next entry should also improve the architecture rather than merely add another fixed iterate. The repository already supplies precisely the needed amplifier: Contender 9 lifts unary F_ω reflection to a forest of oracle families. Applying the same amplifier to the strictly stronger kind-polymorphic language is the lowest-risk, highest-leverage next step. It reuses already proved normalization, checker soundness, family packing, and the self-indexed W-tree while yielding new universal domination theorems.

2 The three reusable ingredients

2.1 Kind-polymorphic reflection

For a unary oracle $f : \mathbb{N} \to \mathbb{N}$, Contender 10 uses

$$\mathrm{reflectFk}(f, n) = n + 1 + \max\left(f(n), \max_{t \in \mathrm{ctmsUpTo}(n)} \mathrm{fvalue}(f, n, t) \right).$$

This exceeds the input, the oracle value at the input, and every accepted kind-polymorphic program enumerated at that budget. The construction is not assumed monotone in f: a program may branch on oracle answers, so pointwise larger oracles need not produce pointwise larger program outputs. All later comparisons therefore proceed by *explicit inclusion* in a finite maximum.

2.2 Packing a family into one oracle

BusyLean already defines an injective code

$$(i, m) \longmapsto 2^i(2m + 1)$$

and proves that its decoder `unpair2` recovers (i, m). Thus a family $fs : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$ is presented to the unary object-language oracle by

$$\mathrm{packFamily}(fs, k) = fs(\mathrm{unpair2}(k)_1, \mathrm{unpair2}(k)_2).$$

The existing theorem `packFamily_code` gives

$$\mathrm{packFamily}(fs, 2^i(2m + 1)) = fs(i, m).$$

No new encoding argument is needed for Contender 11.

2.3 The self-indexed W-tree

The W-language used by Contender 6 denotes countably branching well-founded $W\mathbb{N}$ trees. For every syntax budget k, `treesOfDepth k` joins the trees defined by all programs enumerated at depth at most k; the root `index = .node treesOfDepth` joins all of these levels. The theorem `defined_tree_eventually_lt` for Contender 6, and its Contender 9 analogue, show how to turn membership in the term enumeration into eventual domination by the root hierarchy.

The important distinction is that `index` is not a hand-written ordinal notation. It is generated by the language's own tree-denoting programs. This makes the amplifier self-indexed while preserving structural recursion in Lean.

3 Definition of Contender 11

3.1 Family reflection at one node

For an oracle family $fs : \mathbb{N} \to \mathbb{N} \to \mathbb{N}$, define

$$\begin{aligned} P_{fs}(n) &= \max_{i \le n} fs(i, n), \\ M_{fs}(n) &= \max_{t \in \mathrm{Fk.ctmsUpTo}(n)} \mathrm{Fk.\,fvalue}(\mathrm{packFamily}(fs), n, t), \\ R_{fs}(n) &= n + 1 + \max(P_{fs}(n), M_{fs}(n)). \end{aligned}$$

The Lean definition is direct:

```
def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (Fk.fvalue (packFamily fs) n)
      (Fk.ctmsUpTo n))
```

The explicit prefix maximum makes every child with index $i \le n$ visible even if no object-language program happens to call that child. The program maximum adds the full computational strength of kind-polymorphic F_ω relative to all packed child oracles.

3.2 Reflection along a forest

Reflection is iterated by structural recursion on a W-tree:

```
def stage11 (seed : W.Family) : BigNumber.WTree Nat -> Nat -> Nat
  | .leaf   => reflect11 seed
  | .node g => reflect11 (fun i => stage11 seed (g i))
```

At a leaf the seed family is reflected once. At a node, child i contributes the entire function $n \mapsto \mathrm{stage}_{11}(g(i), n)$ to the parent oracle family. A parent program can therefore choose a child and a child input dynamically through the packing code.

3.3 Seed and diagonal

The predecessor is included as a *family*, not as a constant:

```
def family10 (n : Nat) : Nat := F10 (F10 n)
```

By definitional equality, `family10 42` is `contender_10`. The new hierarchy and number are

```
def F11 (n : Nat) : Nat :=
  stage11 (fun _ => family10) index n

def contender_11 : Nat := F11 (F11 42)
```

The constant `contender_10` does not occur in this dependency graph. The outer diagonal is useful for two independent reasons: the first $F_{11}(42)$ provides a vast syntax/fuel budget, and the second application reflects at that already amplified input.

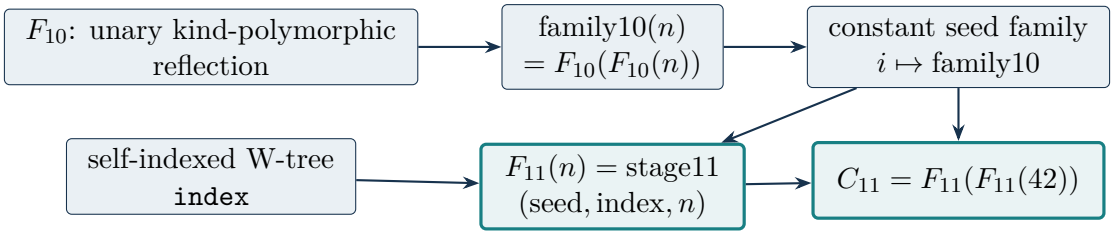


Figure 1: The construction uses the predecessor's generating function as the leaf seed and applies kind-polymorphic reflection at every node of the self-indexed forest.

4 Formal domination theorems

4.1 One-step inequalities

All basic estimates are consequences of membership in one of the two explicit maxima.

Lemma 4.1 (Visible oracle values)**.** *If $i \le n$, then*

$$fs(i, n) < R_{fs}(n).$$

In Lean this is `oracle_lt_reflect11`.

Lemma 4.2 (Visible program values)**.** *If $t \in \mathrm{Fk.ctmsUpTo}(n)$, then*

$$\mathrm{Fk.fvalue}(\mathrm{packFamily}(fs), n, t) < R_{fs}(n).$$

In Lean this is `fvalue_lt_reflect11`.

Lemma 4.3 (Input growth)**.** *For every family fs and input n, $n < R_{fs}(n)$.*

These facts are independent of any monotonicity property of the evaluator. They use only the finite maximum definitions and elementary arithmetic.

4.2 Child-to-parent growth

If $a = \mathrm{node}(g)$ and $i \le n$, then the ith child value appears in the parent's prefix maximum:

$$\mathrm{stage}_{11}(\mathrm{seed}, g(i), n) < \mathrm{stage}_{11}(\mathrm{seed}, a, n).$$

This is `stage11_child_lt`. It is the local edge relation used to embed a chosen W-tree path into the global root hierarchy.

4.3 Pointwise domination of the predecessor family

Theorem 4.4 (`family10_lt_F11`)**.** *For every $n \in \mathbb{N}$,*

$$F_{10}(F_{10}(n)) < F_{11}(n).$$

Proof architecture. The depth-zero term list contains the distinguished zero term, whose denoted tree is proved by the existing theorem `first_tree_leaf` to be a leaf. There are therefore three explicit strict steps:

$$\begin{aligned}\mathrm{family10}(n) &< \mathrm{stage}_{11}(\mathrm{seed}, \mathrm{leaf}, n)\\ &< \mathrm{stage}_{11}(\mathrm{seed}, \mathrm{treesOfDepth}(0), n)\\ &< \mathrm{stage}_{11}(\mathrm{seed}, \mathrm{index}, n) = F_{11}(n).\end{aligned}$$

The first is inclusion of the seed at family index zero; the next two are child-to-parent inequalities at index zero. No evaluator monotonicity is used. □

4.4 The required comparison

Theorem 4.5 (`contender_10_lt_contender_11`)**.**

$$C_{10} < C_{11}.$$

Proof. By definitional equality, $C_{10} = \mathrm{family10}(42)$. The pointwise theorem gives

$$C_{10} < F_{11}(42).$$

Input growth, applied to the whole index tree at input $F_{11}(42)$, gives

$$F_{11}(42) < F_{11}(F_{11}(42)) = C_{11}.$$

Transitivity proves the claim. This is exactly the Lean proof term. □

The certified margin is thus not "old number plus one." The old number lies below the *inner* budget of the new diagonal, and the new hierarchy already pointwise dominates the predecessor's full generating family.

4.5 Program and tree domination

Define the family visible at the root by

$$\mathrm{rootFamily11}(i, n) = \mathrm{stage}_{11}(\lambda_.\mathrm{family10}, \mathrm{treesOfDepth}(i), n).$$

Then Lean proves:

Theorem 4.6 (`fvalue_lt_F11`)**.** *For every $t \in \mathrm{Fk.\,ctmsUpTo}(n)$,*

$$\mathrm{Fk.\,fvalue}(\mathrm{packFamily}(\mathrm{rootFamily11}), n, t) < F_{11}(n).$$

At the final inner budget $n = F_{11}(42)$ this specializes to `fvalue_lt_contender_11`: every enumerated kind-polymorphic program at that budget is below C_{11}.

The index also remains cofinal for the W-language's denoted trees:

Theorem 4.7 (`defined_tree_eventually_lt11`)**.** *If t occurs in the W-language enumeration at depth k, there is N such that for every $n \geq N$,*

$$\mathrm{stage}_{11}(\mathrm{seed}, \mathrm{defTree}(t), n) < F_{11}(n).$$

The witness is explicit: if t has list index i, take $N = k + i$ so both the level k and list position i are visible in the corresponding prefix maxima.

5 Why the number is computable in principle

The value is physically infeasible to compute, but every component is total and effective.

1. **Finite syntax enumeration.** `Fk.ctmsUpTo n` is a finite list generated by primitive recursion on n. The list may be enormous, but it is computable.
2. **Decidable checking.** The Church-style kind-polymorphic checker uses a finite fuel parameter. It may reject a valid term if fuel is insufficient, but accepted terms come with a sound typing derivation.
3. **Certified normalization.** `Fk.sn_of_typed` supplies strong normalization for accepted erased terms with the oracle. Well-founded recursion on that certificate computes a normal form, from which a Church numeral is decoded.
4. **Finite maxima.** `prefixMax` and `maximum` recurse over finite prefixes and lists.
5. **Structural W-recursion.** `stage11` is a structurally accepted recursion on `BigNumber.WTree Nat`; a node recursively exposes each proper child.
6. **Closed final term.** `contender_11` has type `Nat` and no free variables, external functions, custom axioms, noncomputable definitions, unsafe code, or partial recursion.

The proof objects underlying normalization may mention `propext` and `Quot.sound`; these are precisely the axioms allowed by BusyLean. They occur in propositions and do not introduce a computational oracle into the natural-number definition.

6 Size analysis: formal facts and heuristic calibration

6.1 What is formally established

The following chain is kernel-checked:

$$C_{10} = \text{family10}(42) < F_{11}(42) < F_{11}(F_{11}(42)) = C_{11}.$$

More generally,

$$\forall n \quad F_{10}(F_{10}(n)) < F_{11}(n).$$

At every node, $R_{fs}(n)$ simultaneously exceeds all visible child values and all visible F_ω^{kind}-program values relative to the packed child family. Repeating this node operation along `index` is a genuine forest amplifier, not a fixed finite stack written into the source.

6.2 A deliberately cautious heuristic

Let $|F_\omega^{\text{kind}}|$ informally denote the strength of the kind-polymorphic normalizing language, and let ι denote the order type heuristically represented by the W-language self-index. A single reflection step acts like adding one block of $|F_\omega^{\text{kind}}|$ above the oracle's growth rank; iteration along a tree acts like transfinite repetition of that block. The natural informal description is therefore a hierarchy of the shape

$$f_{|F_\omega^{\text{kind}}|\cdot\iota}$$

above a seed already containing two applications of F_{10}. This notation is only an explanatory metaphor. No exact ordinal analysis of the repository's kind-polymorphic system, its oracle extension, or the self-index has been formalized, and this article does not assert one.

The robust conclusion does not depend on the heuristic: the pointwise family comparison, program maximum, child inequalities, and final strict comparison are formal Lean theorems.

Feature	Contender 10	Contender 11
Object language	kind-polymorphic F_ω	same certified language
Oracle interface	one unary oracle	encoded family of child-stage oracles
Amplifier	one reflection at each call to F_{10}	reflection at every node of the self-indexed W-forest
Predecessor use	Contender 9 family as oracle	Contender 10 family as leaf seed
Uniform theorem	`family9 n < F10 n`	`family10 n < F11 n`
Program bound	programs at budget $F_{10}(42)$	programs at budget $F_{11}(42)$ under the root family
Index theorem	none at this layer	every W-definable tree eventually dominated
Final number	$F_{10}(F_{10}(42))$	$F_{11}(F_{11}(42))$

7 Verification and audit record

7.1 Remote kernel/elaborator check

The verification run used Lean 4.34.0 and the exact BusyLean snapshot `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`. The 30 local modules in the transitive dependency closure of `BusyLean.Comparison10` were ordered topologically, concatenated after removing only their import lines, and followed by the exact two source modules in this archive. The resulting unit was submitted to AXLE's `check` service with unbounded heartbeats and a 300 second request limit.

The recorded result was:

Field	Result
Lean environment	`lean-4.34.0`
Compilation	`okay = true`
Failed declarations	empty list
Lean errors	none
Combined check time	8.900 seconds
Axioms of five principal new declarations	`[propext, Quot.sound]`
Forbidden predecessor constants in C_{11} closure	none
Transitive closure size reported by audit	1,460 constants

The five printed declarations were `contender_11`, the required comparison, the pointwise family theorem, the W-tree eventual-domination theorem, and the final program-bound theorem. AXLE also returned an empty validation-failure list, so no `sorry`, unsafe declaration, or disallowed axiom was detected.

7.2 What this check does and does not establish

The merged verification is strong evidence because Lean parses, elaborates, and kernel-checks the exact declarations against the exact dependency source. It also catches theorem holes, type errors, termination errors, and axiom-policy violations. It is not identical to a clean Lake build of the original module graph: module import boundaries, generated object files, and the repository's wall-clock budget harness are not exercised by a one-file merge. Therefore the archive includes `scripts/check_contender11.py`, which performs the official staged build after integration:

1. prebuild `BusyLean.Comparison10` as accepted history;

2. time `BusyLean.Contender11` against 15 seconds;

3. time `BusyLean.Comparison11` against 60 seconds;

4. run `test/Contender11Audit.lean`.

Given that the entire merged current stack plus the new proof took under nine seconds on the remote checker, the added modules are expected to fit comfortably, but the supplied local script is the authoritative budget test.

8 Engineering choices and rejected alternatives

8.1 Why explicit inclusion beats monotonicity

It is tempting to claim that replacing an oracle by a larger oracle can only increase a program's result. That is false: a program can compare an oracle answer with a numeral and take a smaller branch when the answer grows. The proof therefore identifies concrete entries in `prefixMax` and `maximum`. This method is robust under arbitrary oracle-sensitive control flow and matches the repository's existing Contender 9 proof style.

8.2 Why reuse the W-language index

One might ask for an index generated by the kind-polymorphic language itself. Its Church programs return natural numbers, not Lean W-trees, and the normalization proof deliberately operates on erased syntax rather than a set-theoretic model capable of returning infinitary trees. The existing W-language already supplies a certified universe of tree schedules and a self-index theorem. Reusing it isolates the new contribution to the stronger reflection layer.

8.3 Why not jump immediately to a kind-universe ladder

The repository roadmap proposes stratified universes of kinds, allowing kinds to quantify over lower-level kinds while placing the result one level higher. That would be a genuine language-strength extension and may eventually dwarf this proposal. It also requires new syntax, kinding, substitution, conversion, semantic domains, candidates, checker proofs, enumeration, and audit work. Forest reflection over `Fk` is already available conceptually and can be proved with a small, reviewable delta. Contender 11 is therefore a strong conservative extension and a clean staging point before a universe-ladder Contender 12.

8.4 Why not use the incumbent constant

Defining $C_{11} = G(C_{10})$ would make a numerical comparison easy but would violate BusyLean's anti-laziness rule. The present definition uses F_{10} as a reusable construction, forms the family $n \mapsto F_{10}(F_{10}(n))$, and subjects that family to a qualitatively stronger forest reflection. The transitive dependency audit mechanically enforces this distinction.

9 Further research questions

R1. Generic forest-reflection library. Abstract the repeated Contender 9/11 pattern over a record containing a program enumeration, evaluator, unary-oracle packer, and domination lemmas. This could reduce future contenders to an instantiation plus a small proof.

R2. Kind-universe ladder. Formalize levels $\square_0 \subset \square_1 \subset \cdots$ above the impredicative kind $\star$, prove normalization level by level, and diagonalize over the number of levels used by a program.

R3. Large elimination. Add runtime-to-type dependency with certified large elimination, approaching a Calculus of Inductive Constructions style jump while retaining a total executable normalizer.

R4. Richer schedules. Design a total language whose programs denote schedules for the kind-polymorphic reflection itself, rather than importing the older W-language index. The central issue is a safe semantic interface from programs to well-founded trees.

R5. Formal ordinal lower bounds. Define an internal fast-growing hierarchy and prove explicit embeddings of notation systems into `stage11`. Even a conservative lower bound would turn the current heuristic size discussion into a theorem.

R6. Checker completeness envelopes. Quantify how much the fuel-bounded kind/type normalizer omits and prove that standard families of high-growth programs are accepted within explicit fuel bounds.

R7. Independent normalization audit. Cross-check `Fk.sn_of_typed` with a second logical relation or a translation into an independently formalized normalizing calculus.

R8. Build optimization. Profile enumeration elaboration and generated code so that future language extensions remain comfortably inside the 15 second definition budget.

R9. No-`Quot.sound` variant. Investigate whether proof engineering can remove function extensionality dependencies from the new comparison layer, even if the inherited normalization development still uses them.

R10. Stronger diagonal policies. Compare $F_{11}(F_{11}(42))$ with principled multi-parameter diagonals that separately grow the syntax budget, tree visibility bound, and child input, while avoiding arbitrary padding.

10 Reproducibility and archive layout

The archive is organized for direct review and integration:

Path	Purpose
`BusyLean/Contender11.lean`	Definitions of `reflect11`, `stage11`, `family10`, `rootFamily11`, `F11`, and `contender_11`.
`BusyLean/Comparison11.lean`	One-step inequalities, pointwise family comparison, required strict comparison, tree cofinality, and program bounds.
`test/Contender11Audit.lean`	Axiom reports, predecessor-constant closure audit, and declaration checks.
`scripts/check_contender11.py`	Staged clean build and official contest budget check after integration.
`scripts/verify_axle.py`	Standard-library-only reproducer for the pinned merged Lean 4.34.0 check.
`patches/0001-contender11.patch`	Unified patch adding the modules and updating the umbrella module, main audit, and timing configuration.

Path	Purpose
`verification/AXLE_REPORT.*`	Machine-readable and human-readable record of the remote verification.
`article/contender11.tex`, `.pdf`	This research article and compiled PDF.
`MANIFEST.sha256`	SHA-256 digest of every packaged artifact.

To integrate, copy the two Lean modules, append their imports to `BusyLean.lean`, copy the standalone audit and timing script, then run `python3 scripts/check_contender11.py`. At no point should any contender be evaluated.

11 Conclusion

Contender 11 is a compact but genuine architectural advance over the current BusyLean entry. It preserves the strongest certified object language already available—kind-polymorphic System F_ω—and adds the repository's strongest reusable amplifier: dynamically indexed family reflection iterated along a self-indexed W-forest. The result is closed and computable in principle, uses only the allowed axioms, avoids every predecessor constant, pointwise beats the predecessor's generating family, dominates all visible programs at its final inner budget, and comes with a kernel-checked theorem $C_{10} < C_{11}$.

The design is intentionally conservative about unproved ordinal claims and aggressive about formal auditability. It is suitable for immediate integration as Contender 11 and provides a clean base for the more ambitious kind-universe and large-elimination programs suggested by the repository roadmap.

References

[1] BusyLean contributors. *BusyLean: name the biggest number, in Lean.* Repository snapshot `4c4b03d0e2003c479e3bee90a21c8bea13f66c40`, October 2026.

[2] Jean-Yves Girard. *Interprétation fonctionnelle et èlimination des coupures de l'arithmétique d'ordre supérieur.* Thèse d'État, Université Paris VII, 1972.

[3] Henk P. Barendregt. Lambda calculi with types. In S. Abramsky, D. M. Gabbay, and T. S. E. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, pages 117–309. Oxford University Press, 1992.

[4] Jimmy Xin, Alex Schneidman, Chris Cummins, Karun Ram, Srihari Ganesh, and Jannis Limperg. AXLE: A cloud infrastructure for Lean 4 theorem proving utilities. arXiv:2606.26442, 2026.

A Complete Lean definition module

```
import BusyLean.Contender10

/-!
# Contender 11: forest reflection of kind-polymorphic System Fω

This lifts Contender 10's kind-polymorphic reflection from a unary oracle to
a dynamically indexed oracle family and iterates it along the W-language's
self-indexed tree `index`.
-/
```

```
namespace BusyLean

/-- One reflection step over kind-polymorphic System Fω with an oracle family. -/
def reflect11 (fs : W.Family) (n : Nat) : Nat :=
  n + 1 + BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
    (BigNumber.maximum (Fk.fvalue (packFamily fs) n) (Fk.ctmsUpTo n))

/-- Forest reflection of kind-polymorphic System Fω along a W-tree. -/
def stage11 (seed : W.Family) : BigNumber.WTree Nat → Nat → Nat
  | .leaf => reflect11 seed
  | .node g => reflect11 (fun i => stage11 seed (g i))

/-- Contender 10's construction as a function of its input. -/
def family10 (n : Nat) : Nat := F10 (F10 n)

/-- The oracle family visible at the root of `index`. -/
def rootFamily11 : W.Family :=
  fun i => stage11 (fun _ => family10) (treesOfDepth i)

/-- Kind-polymorphic forest reflection along the W-language's self-index. -/
def F11 (n : Nat) : Nat := stage11 (fun _ => family10) index n

/-- **Contender 11.** Never evaluate it. -/
def contender_11 : Nat := F11 (F11 42)

end BusyLean
```

B Complete Lean comparison module

```
import BusyLean.Contender11
import BusyLean.Comparison10

/-! # Contender 11: comparison theorems -/

namespace BusyLean

/-! ## Reflection inequalities -/

theorem lt_reflect11 {fs : W.Family} {n m : Nat}
    (h : m ≤ BigNumber.maxN (W.prefixMax (fun i => fs i n) n)
      (BigNumber.maximum (Fk.fvalue (packFamily fs) n) (Fk.ctmsUpTo n))) :
    m < reflect11 fs n := by
  unfold reflect11
  omega

theorem oracle_lt_reflect11 (fs : W.Family) {i n : Nat} (hi : i ≤ n) :
    fs i n < reflect11 fs n :=
  lt_reflect11 (Nat.le_trans (W.le_prefixMax (fun j => fs j n) hi)
    (BigNumber.le_maxN_left _ _))

theorem fvalue_lt_reflect11 (fs : W.Family) {n : Nat} {t : Fk.CTm}
    (ht : t ∈ Fk.ctmsUpTo n) :
    Fk.fvalue (packFamily fs) n t < reflect11 fs n :=
  lt_reflect11 (Nat.le_trans (W.le_maximum _ ht)
    (BigNumber.le_maxN_right _ _))

theorem input_lt_reflect11 (fs : W.Family) (n : Nat) :
```

```
    n < reflect11 fs n := by
  unfold reflect11
  omega

theorem stage11_child_lt (seed : W.Family)
    (g : Nat → BigNumber.WTree Nat) {i n : Nat} (hi : i ≤ n) :
    stage11 seed (g i) n < stage11 seed (.node g) n :=
  oracle_lt_reflect11 (fun j => stage11 seed (g j)) hi

theorem input_lt_stage11 (seed : W.Family)
    (a : BigNumber.WTree Nat) (n : Nat) : n < stage11 seed a n := by
  cases a with
  | leaf => exact input_lt_reflect11 seed n
  | node g => exact input_lt_reflect11 _ n

/-! ## Contender 11 -/

/-- At every input, `F11` exceeds Contender 10's entire generating family. -/
theorem family10_lt_F11 (n : Nat) : family10 n < F11 n := by
  have h₁ : family10 n < stage11 (fun _ => family10) .leaf n :=
    oracle_lt_reflect11 (fun _ => family10) (Nat.zero_le n)
  have h₂ :
      stage11 (fun _ => family10)
          (defTree (nthD (BigNumber.allTerms 0) 0 .zero)) n <
        stage11 (fun _ => family10) (treesOfDepth 0) n :=
    stage11_child_lt _
      (fun j => defTree (nthD (BigNumber.allTerms 0) j .zero))
      (Nat.zero_le n)
  have h₃ : stage11 (fun _ => family10) (treesOfDepth 0) n <
      stage11 (fun _ => family10) index n :=
    stage11_child_lt _ treesOfDepth (Nat.zero_le n)
  rw [first_tree_leaf] at h₂
  exact Nat.lt_trans h₁ (Nat.lt_trans h₂ h₃)

/-- **The required comparison**: Contender 10 is strictly smaller. -/
theorem contender_10_lt_contender_11 : contender_10 < contender_11 :=
  Nat.lt_trans (family10_lt_F11 42)
    (input_lt_stage11 _ index (F11 42))

/-- Every tree defined by the W-language is eventually dominated, with
kind-polymorphic reflection at every node. -/
theorem defined_tree_eventually_lt11 {k : Nat} {t : BigNumber.Term}
    (ht : t ∈ BigNumber.allTerms k) :
    ∃ N, ∀ n, N ≤ n →
      stage11 (fun _ => family10) (defTree t) n < F11 n := by
  obtain ⟨i, hi⟩ := exists_nthD_of_mem (d := BigNumber.Term.zero) ht
  refine ⟨k + i, fun n hn => ?_⟩
  rw [← hi]
  exact Nat.lt_trans
    (stage11_child_lt _
      (fun j => defTree (nthD (BigNumber.allTerms k) j .zero)) (by omega))
    (stage11_child_lt _ treesOfDepth (by omega))

/-- Every kind-polymorphic program visible at budget `n` is below `F11 n`. -/
theorem fvalue_lt_F11 {n : Nat} {t : Fk.CTm}
    (ht : t ∈ Fk.ctmsUpTo n) :
    Fk.fvalue (packFamily rootFamily11) n t < F11 n := by
  unfold F11 index rootFamily11
  exact fvalue_lt_reflect11 _ ht
```

```
/-- Every kind-polymorphic program at the final inner budget is exceeded. -/
theorem fvalue_lt_contender_11 {t : Fk.CTm}
    (ht : t ∈ Fk.ctmsUpTo (F11 42)) :
    Fk.fvalue (packFamily rootFamily11) (F11 42) t < contender_11 :=
  fvalue_lt_F11 ht

theorem contender_9_lt_contender_11 : contender_9 < contender_11 :=
  Nat.lt_trans contender_9_lt_contender_10 contender_10_lt_contender_11

end BusyLean
```